

Weekly Report

General Information

- **Group ID:** 55
- **Project Name:** CS202 - Apple Knight
- **GitHub Repository:** <https://github.com/kietanhnguyen2007/CS202-FinalProject>
- **Date range:** 29/6/2026 - 3/7/2026

Detailed Tasks Completed This Week

While our Markdown report provides a high-level summary of the week's progress, this document elaborates on the technical implementations and architectural decisions made by each member.

25125074 – Nguyen Anh Kiet

- **MVC Architecture Setup:** Engineered the foundational structure of the game by strictly enforcing the Model-View-Controller pattern. This involved creating isolated directories (`include/Model`, `include/View`, `include/Controller`) and ensuring that the business logic (Model) has zero dependencies on the rendering engine. This modularity allows the game logic to be tested independently and simulated on a headless server.
- **Core Data Models:** Designed the primary entity hierarchy. Created an abstract `Entity` base class from which `Character`, `Player`, `Enemy`, and `Boss` inherit. Additionally, implemented the `GameState` class which acts as the central data repository for the current level, managing lists of active entities and player statistics.
- **Game & Menu Controllers:** Developed the central orchestrators for the application. `GameController` manages the fixed-timestep physics update loop, gathers player inputs via `InputController`, updates the Model, and subsequently commands the View layer to render the updated state.
- **Authoritative Multiplayer Networking:** Integrated the ENet library to build a reliable UDP communication layer. Implemented an Authoritative Server architecture where the server computes the absolute truth of the game state (validating movement and combat) to prevent client-side manipulation. Designed a custom `Packet` serialization system to efficiently transmit entity states, health updates, and input commands across the network.
- **Elemental Reaction System:** Developed a complex status effect system inspired by modern RPGs. When a player applies a specific element (e.g., Fire) to an enemy already suffering from another element (e.g., Water), the `ElementalSystem` calculates and triggers a specific reaction (e.g., Vaporize for bonus damage). This logic is synchronized across the server to ensure consistency for all clients.

25125037 – Nguyen Trong Tien

- **Raylib Graphics Integration:** Initialized the Raylib context and constructed a Singleton `Renderer` class. This class acts as a wrapper around Raylib's drawing functions, allowing the View layer to batch sprite and primitive drawing calls efficiently while hiding the underlying graphics API from the rest of the game.
- **UI Framework & Components:** Designed a decoupled User Interface system. Created specific view classes such as `HUDView` (for health bars and skill cooldowns), `MenuView`, and `GameView`. These classes read passively from the Model and render the visual state without altering game data.
- **Dynamic UI Scaling (`WindowManager`):** Solved the issue of UI distortion on varying monitor resolutions by developing a `WindowManager` singleton. This system hooks into the main loop to detect window resize events, queries the OS for the current monitor's DPI factor, and calculates a uniform, letterbox-safe scaling multiplier (`min(scaleX, scaleY)`). All UI components now use this multiplier to remain perfectly proportioned.
- **Quadtree Spatial Partitioning:** Optimized the collision detection system, which previously suffered from $O(n^2)$ time complexity. Developed a `Quadtree` data structure that recursively subdivides the 2D space. The game now only checks for collisions between entities located in the same or adjacent spatial sectors, drastically reducing computational overhead to $O(n \log n)$.
- **Zero-Allocation Object Pool:** Addressed severe garbage collection lag spikes by implementing the `ObjectPool` design pattern. Projectiles and particles are now pre-allocated in a fixed block of memory during the level loading phase. When required, they are activated rather than instantiated via `new`, ensuring zero runtime dynamic memory allocation during gameplay.
- **Particle System:** Created a dynamic visual effects manager that handles the lifecycle of short-lived visual entities (like explosions and sparks), tightly integrated with the Object Pool for maximum performance.

AI Usage Declaration

- **Debugging & Constraint Enforcement:** AI was utilized to trace memory leaks and enforce our strict "Zero Runtime Allocation" constraint. When dynamic allocations were detected inside the main loop, AI assisted in refactoring the code to utilize pre-allocated memory pools.
- **Architectural Blueprints:** AI provided mathematical and structural suggestions for the `WindowManager`'s dynamic DPI scaling logic, ensuring the calculations correctly accounted for aspect ratio preservation (letterboxing).

Tasks Planned for Next Week

- Polish multiplayer synchronization to implement client-side prediction, masking network latency for a smoother player experience.

- Integrate the **LevelFactory** to load complex map layouts from external data files, supporting multi-layered parallax backgrounds.
- Implement the **SoundManager** to handle spatial audio and background music streaming.
- Refine boss AI behaviors for co-op mode, ensuring bosses react dynamically to the presence of multiple players.

Issues & Resolutions

- **Issue 1:** Performance degradation (stuttering) occurred when spawning numerous projectiles simultaneously.
Resolution: Replaced dynamic `new/delete` operations with a pre-allocated `ObjectPool`, eliminating runtime memory allocation overhead.
- **Issue 2:** UI elements became misaligned or stretched when the game window was resized.
Resolution: Engineered the **WindowManager** to recalculate a universal UI scale factor dynamically based on the window's aspect ratio and monitor DPI, applying this scale to all rendering coordinates and font sizes.